

ECE0202 Lecture 20

Serial Peripheral Interface (SPI) 中文逐页详细讲义

根据课堂 PPT 整理

June 8, 2026

关键结论

本讲的主线只有一条：**SPI 是一种由 master 提供时钟、靠 SCK/MOSI/MISO/NSS 四类信号进行同步串行通信的接口**。学习时不要死背寄存器名，要先抓住数据怎么流、时钟谁给、什么时候采样、什么时候发送、状态位如何告诉你能不能读写。

目录

1	Slide 1: 封面 - 本讲进入 SPI	2
2	Slide 2: Topics - 本讲内容结构	2
3	Slide 3: Serial Peripheral Interface (SPI) 在 STM32L4 中的位置	2
4	Slide 4: Application Examples - SPI 的应用场景	2
5	Slide 5: SPI 的基本特点	3
6	Slide 6: Block Diagram - SPI 四根核心信号线	3
7	Slide 7: Interconnection - Full-duplex 全双工 SPI	4
8	Slide 8: Interconnection - Half-duplex 半双工 SPI	4
9	Slide 9: Interconnection - Simplex 单向 SPI	5
10	Slide 10: Multi-slave - 星形拓扑和片选	5
11	Slide 11: Data Exchange - SPI 数据交换的本质	5
12	Slide 12: Data Frame Size - 数据帧长度 4 到 16 bit	6
13	Slide 13: Clock - SPI 时钟频率由 prescaler 决定	6
14	Slide 14: Clock - Bit Order: MSB First 或 LSB First	6
15	Slide 15: Clock - Sampling Edge 与 Shifting Edge	7
16	Slide 16: Timing Diagram Example - 时序图怎么读	7

17 Slide 17: Clock Modes - CPOL 和 CPHA	7
18 Slide 18: Timing Diagram - CPOL=0, CPHA=1, LSB First=1	8
19 Slide 19: How to Program SPI - 编程总览	8
20 Slide 20: Block Diagram - SPIx_DR 数据寄存器	9
21 Slide 21: SPIx_DR 的读写含义	9
22 Slide 22: RXNE 与 FRLVL - 接收 FIFO 什么时候能读	9
23 Slide 23: SPIx_CR2 - 数据长度和 FIFO 阈值	9
24 Slide 24: TXE - 发送 FIFO 什么时候能写	10
25 Slide 25: FTLVL 和 TXE - 发送 FIFO 占用水平	10
26 Slide 26: BSY - SPI 忙标志	10
27 Slide 27: Example - STM32L4 连接 L3GD20 Gyro	11
28 Slide 28: Processor's Pins - 从芯片引脚表找 SPI 功能	11
29 Slide 29: Alternate Mode - 把 GPIO 引脚设置为复用功能	11
30 Slide 30: Which Alternate Mode - AFRL 低 8 个引脚	12
31 Slide 31: Which Alternate Mode - AFRH 高 8 个引脚	12
32 Slide 32: SPI Initialization - 开 SPI 时钟并复位	12
33 Slide 33: SPI Initialization - 通信模式与数据帧格式	13
34 Slide 34: SPI Initialization - bit order, CPHA, CPOL, baud rate	13
35 Slide 35: SPI Initialization - master, RXNE 阈值, enable SPI	14
36 Slide 36: SPI Write - 写数据函数逐行解释	14
37 Slide 37: SPI Read - 为什么读也要写 dummy byte	15
38 Slide 38: Interrupt - SPI 也可以用中断	15
39 Slide 39: Polling vs Interrupt - SPI 中断使能路径	15
40 Slide 40: Interrupt Handler - SPI 中断服务函数	16
总复习: Lecture 20 必须掌握的知识点	17

1 Slide 1: 封面 - 本讲进入 SPI

这一页只是封面，说明本讲属于 ECE0202 Embedded Processors & Interfacing + Lab，主题是 SPI。前面 Lecture 19 讲的是 UART，UART 是**异步通信 asynchronous communication**；本讲 SPI 是**同步通信 synchronous communication**。

核心词汇

- **SPI**: Serial Peripheral Interface, 串行外设接口。
- **Serial**: 串行，数据一 bit 一 bit 排队传，不是很多 bit 同时传。
- **Peripheral**: 外设，比如传感器、显示屏、存储芯片。
- **Interface**: 接口，规定两边怎么接线、怎么传数据、什么时候读数据。

2 Slide 2: Topics - 本讲内容结构

PPT 列出三部分：

1. **Serial Peripheral Interface (SPI)**: 先讲 SPI 是什么，线怎么连，数据怎么交换。
2. **How to Program SPI**: 再讲 STM32L4 里要配置哪些寄存器，例如 SPIx_CR1, SPIx_CR2, SPIx_SR, SPIx_DR。
3. **Example**: 最后用 STM32L4 连接一个 SPI 从设备做例子。

关键结论

本讲的学习顺序应该是：先理解**通信规则**，再理解**寄存器配置**，最后理解**读写代码**。如果反过来直接背代码，很容易把 TXE、RXNE、BSY 这些状态位背成一团浆糊。

3 Slide 3: Serial Peripheral Interface (SPI) 在 STM32L4 中的位置

这一页把 SPI 放到 STM32L4 的系统框图里。STM32L4 有 CPU、Flash、SRAM、GPIO、Timer、UART、SPI 等模块。SPI 是挂在总线系统上的一个外设模块。

PPT 标出：STM32L4 有三个 SPI: SPI1, SPI2, SPI3。这表示芯片内部有多个独立 SPI 控制器，你可以根据引脚和外设需求选择使用哪个。

关键概念：SPIx

SPIx 里的 x 是占位符。SPI1, SPI2, SPI3 都可以写成 SPIx 来统一描述。比如 SPIx_CR1 就表示“某一个 SPI 的控制寄存器 1”。

关键结论

SPI 不是 CPU 指令，也不是普通 GPIO。它是 STM32 内部的一个**硬件通信外设**。CPU 通过写 SPI 的寄存器，让 SPI 硬件自动产生时钟、移位发送、接收数据。

4 Slide 4: Application Examples - SPI 的应用场景

这一页列出 SPI 的常见用途，例如 display、smart sensors、memories、MMC/SD cards、I/O expanders。

为什么这些设备喜欢用 SPI？原因很直接：SPI 速度快，硬件简单，控制明确。比如：

- **显示屏 display**: 需要快速传像素数据。

- **传感器 sensor**: 比如陀螺仪、加速度计、温度传感器, 需要主控读取寄存器。
- **Flash memory / EEPROM**: 主控要读写存储数据。
- **SD card**: 可以用 SPI 模式进行简化连接。
- **I/O expander**: 用少量 MCU 引脚扩展更多输入输出。

易错点

SPI 的优势是快、简单, 但代价是线比较多, 而且没有像 UART 那样只用 TX/RX/GND 就能通信。典型 SPI 至少要 SCK、MOSI、MISO, 再加片选 NSS/CS。

5 Slide 5: SPI 的基本特点

这一页列出 SPI 的四个基本性质。

1. Synchronous communication - 同步通信

Synchronous 表示通信双方共享一个时钟。SPI 里由 master 提供 **SCK**, slave 跟着这个时钟发送/接收 bit。

这和 UART 最大不同。UART 没有单独 clock 线, 双方靠约定 baud rate; SPI 有 clock 线, 所以每一 bit 在什么时候传是由 SCK 明确规定的。

2. Can have multiple slave devices - 可以连接多个从设备

一个 master 可以连接多个 slave。SCK、MOSI、MISO 可以共用, 但每个 slave 通常要有单独的片选线 **NSS/CS**。

3. No flow control or acknowledgment - 没有流控或确认

SPI 本身没有“我收到啦”的 ACK 机制, 也没有“我现在太忙别发”的 flow control 机制。master 一给时钟, 数据就开始移动。

4. Slave cannot communicate with slave directly - 从设备不能直接互相通信

SPI 网络中, master 是中心。slave 一般只响应 master, 不会 slave A 直接找 slave B 通信。

关键结论

SPI 的基本逻辑: master 选择一个 slave, master 提供 clock, 双方按 clock 同时移位交换数据。

6 Slide 6: Block Diagram - SPI 四根核心信号线

这一页给出 SPI 的核心连线: **SCK, MOSI, MISO, NSS**。

信号线	英文全称	中文含义
SCK	Serial Clock	串行时钟, 由 master 输出
MOSI	Master Out Slave In	主出从入, master 发给 slave
MISO	Master In Slave Out	主入从出, slave 发给 master
NSS/CS	Slave Select / Chip Select	片选, 通常低电平有效

为什么叫 MOSI/MISO?

名字已经把方向写死了:

- **MOSI**: master 的输出, slave 的输入。
- **MISO**: master 的输入, slave 的输出。

NSS active low 是什么意思?

active low 表示低电平有效。也就是:

$NSS = 0 \Rightarrow$ 这个 slave 被选中

$NSS = 1 \Rightarrow$ 这个 slave 不参与通信

易错点

SPI 接线最常见错误: MOSI/MISO 接反, 或者忘记把 CS/NSS 拉低。CS 没选中时, slave 通常不会理你, 像你给一个没接电话的人讲话。

7 Slide 7: Interconnection - Full-duplex 全双工 SPI

这一页讲默认 SPI 连接方式: **full-duplex**。

Full-duplex 中文是全双工, 意思是 master 和 slave 可以同时发送、同时接收。SPI 能做到这一点, 是因为 MOSI 和 MISO 是两条不同方向的线。

方向	使用线路	数据流向
Master 发给 Slave	MOSI	Master $\rightarrow$ Slave
Slave 发给 Master	MISO	Slave $\rightarrow$ Master
Clock	SCK	Master $\rightarrow$ Slave
片选	NSS/CS	Master $\rightarrow$ Slave

每一个 SCK 脉冲, master 通过 MOSI 推出一个 bit, 同时 slave 通过 MISO 推出一个 bit。两边不是“先发后收”, 而是边发边收。

关键结论

SPI 的 full-duplex 本质是**交换数据 exchange**, 不是单纯发送。你写 SPIx_DR 发出一个 byte 的同时, 硬件也会从 MISO 收进一个 byte。

8 Slide 8: Interconnection - Half-duplex 半双工 SPI

这一页讲 **half-duplex**, 中文叫半双工。

半双工的意思是: 双方都能发送和接收, 但**同一时刻只能一个方向传**。PPT 里说节点共享一根 common single data line, 也就是用一根数据线轮流收发。

模式	数据线数量	同时收发?
Full-duplex	MOSI + MISO 两根	可以
Half-duplex	共用一根数据线	不可以, 需要轮流

STM32 里通过 SPIx_CR1 的 BIDIMODE 等位选择 bidirectional mode。

易错点

Half-duplex 不是“低级版全双工”，而是线更少、控制更复杂。共用一根线时，必须避免 master 和 slave 同时驱动这根线，否则可能发生总线冲突。

9 Slide 9: Interconnection - Simplex 单向 SPI

这一页讲 **simplex**。Simplex 中文可译为单工或单向通信。

单向通信表示数据只沿一个方向传。比如 master 只发不收，或者 master 只收不发。PPT 中提到 unidirectional mode，表示发送方向和接收方向被固定。

Simplex 与 Half-duplex 的区别

- **Simplex**: 方向固定，只能一个方向。
- **Half-duplex**: 两个方向都可以，但不能同时，要轮流。
- **Full-duplex**: 两个方向可以同时。

10 Slide 10: Multi-slave - 星形拓扑和片选

这一页讲多个 slave 的 SPI 网络。PPT 写的是 **star topology**，中文是星形拓扑。

一个 master 可以接多个 slave。常见做法：

- SCK 共用。
- MOSI 共用。
- MISO 共用，但未选中的 slave 必须高阻或不驱动。
- 每个 slave 有独立的 CS/NSS 线。

为什么每个 slave 要单独 CS?

因为 master 必须告诉总线上所有 slave：“现在我只和你说话”。如果多个 slave 同时响应，就会造成 MISO 总线冲突。

关键结论

多 slave SPI 的关键不是 SCK/MOSI/MISO，而是片选管理 **CS management**。每次通信前先拉低目标 slave 的 CS，通信结束后拉高。

11 Slide 11: Data Exchange - SPI 数据交换的本质

这一页讲 SPI 数据交换。重点有三句：master 提供 clock；每个 clock pulse 移出一 bit、移入一 bit；只有 master 能开始传输。

SPI 内部可以想成两个移位寄存器：master 一个，slave 一个。每来一个 SCK：

- master 把自己的最高位或最低位推出到 MOSI。
- slave 在 MISO 上推出自己的一个 bit。
- 双方同时把对方线上当前 bit 移入自己的接收寄存器。

关键结论

SPI 不是“发送时只发送，接收时只接收”。在 full-duplex 模式下，每个时钟都是**交换 swap**：送出去一个 bit，也收回来一个 bit。

12 Slide 12: Data Frame Size - 数据帧长度 4 到 16 bit

这一页讲 data frame size。Data frame 可以理解成 SPI 一次完整传输的数据单位。STM32L4 中 SPI frame size 可以是 4 到 16 bits。

SPIx_CR2 里的 DS[3:0] 控制 frame length。比如常用 8-bit frame，因为很多外设寄存器都是 byte 为单位。

Frame 与 Bit 的关系

- **Bit**: 最小单位，一个 0 或 1。
- **Frame**: 一组 bit，例如 8-bit frame 就是一次传 8 个 bit。
- **Byte**: 8 bit。很多 SPI 设备按 byte 读写。

易错点

Master 和 slave 必须对 frame size 有一致理解。你用 8-bit frame 和一个期望 16-bit frame 的设备通信，数据边界会错位，结果读出来像乱码。

13 Slide 13: Clock - SPI 时钟频率由 prescaler 决定

这一页开始讲 SPI clock。SPI 的 SCK 不是凭空产生的，它来自 STM32 的外设时钟 f_PCLK，再经过 SPI baud rate prescaler 分频。

一般关系可以理解为：

$$f_{SCK} = \frac{f_{PCLK}}{\text{prescaler}}$$

PPT 的 SPIx_CR1 中有 BR[2:0] 位控制分频系数，常见从 $f_{PCLK}/2$ 到 $f_{PCLK}/256$ 。

关键结论

SPI 速度不是由 slave 主动决定，而是 master 输出 SCK 决定。slave 只能跟着 master 的时钟工作。如果 SCK 太快，slave 来不及响应，就会读错数据。

14 Slide 14: Clock - Bit Order: MSB First 或 LSB First

这一页指出 SPI 可以选择先传最高位还是先传最低位。对应控制位通常是 LSBFIRST。

- **MSB first**: Most Significant Bit first, 最高有效位先传。
- **LSB first**: Least Significant Bit first, 最低有效位先传。

例如一个 byte 是：

0b1011_0010

MSB first 时传输顺序是：

1, 0, 1, 1, 0, 0, 1, 0

LSB first 时传输顺序是：

0, 1, 0, 0, 1, 1, 0, 1

易错点

Bit order 必须和 slave datasheet 匹配。多数 SPI 设备使用 MSB first, 但不能靠猜。嵌入式里“我以为默认一样”就是 bug 工厂。

15 Slide 15: Clock - Sampling Edge 与 Shifting Edge

这一页是 SPI 最容易卡住的地方。PPT 标出两个概念: **Sampling/Capture Edge** 和 **Shifting/Toggling Edge**。

边沿 edge

- **Rising edge**: 上升沿, SCK 从 0 变 1。
- **Falling edge**: 下降沿, SCK 从 1 变 0。

Sampling/Capture Edge - 采样边沿

接收方在这个边沿读取数据线上的电平, 把它当成本 bit 的值。

Shifting/Toggling Edge - 移位/翻转边沿

发送方在这个边沿改变数据线上的值, 为下一个 bit 做准备。

关键结论

原则: 发送方在一个边沿改变数据, 接收方在另一个边沿采样数据。这样数据在采样时已经稳定, 不会刚变化就被读错。

16 Slide 16: Timing Diagram Example - 时序图怎么读

这一页给出一个时序图。读 SPI 时序图时按下面顺序看:

1. 先看 SCK 空闲时是 0 还是 1, 也就是 CPOL。
2. 再看数据在哪个边沿被采样, 也就是 CPHA。
3. 再看 MOSI/MISO 数据位顺序, 是 MSB first 还是 LSB first。
4. 最后看 CS/NSS 什么时候拉低、什么时候拉高。

易错点

不要只盯 MOSI/MISO 的 0 和 1。SPI 的核心是数据在什么时刻有效。同样一串波形, 如果 CPHA/CPOL 理解错, 采样点错了, 读出的 bit 就会整体偏移。

17 Slide 17: Clock Modes - CPOL 和 CPHA

这一页总结 SPI 的四种 clock mode。它们由两个 bit 决定: CPOL 和 CPHA。

Mode	CPOL	CPHA	直白含义
0	0	0	空闲低, 第一个边沿采样
1	0	1	空闲低, 第二个边沿采样
2	1	0	空闲高, 第一个边沿采样
3	1	1	空闲高, 第二个边沿采样

CPOL - Clock Polarity 时钟极性

CPOL 决定 SCK 空闲状态:

$CPOL = 0 \Rightarrow SCK$ 空闲为低

$CPOL = 1 \Rightarrow SCK$ 空闲为高

CPHA - Clock Phase 时钟相位

CPHA 决定第几个时钟边沿采样:

$CPHA = 0 \Rightarrow$ 第一个边沿采样

$CPHA = 1 \Rightarrow$ 第二个边沿采样

关键结论

CPOL 决定“时钟平时待在哪个电平”，CPHA 决定“第几个边沿读数据”。这两个必须和 slave 的 datasheet 完全一致。

18 Slide 18: Timing Diagram - CPOL=0, CPHA=1, LSB First=1

这一页给出具体时序: CPOL=0, CPHA=1, LSB First=1。

逐项解释:

- CPOL=0: SCK 空闲时是低电平。
- CPHA=1: 第一次 transition 不采样, 第二次 transition 才采样。
- LSB First=1: 最低位先传。

假设发送数据 0b10110010, LSB first 意味着先传最右边的 0, 再传 1, 再传 0……而不是从最高位 1 开始。

易错点

时序图里最容易错的是把 CPHA=1 理解成“第二个 bit 才开始”。不是。CPHA=1 是第一个 bit 在第二个边沿采样, 不是丢掉第一个 bit。

19 Slide 19: How to Program SPI - 编程总览

这一页是过渡页, 说明后面开始讲如何配置 SPI。

配置 SPI 通常分成几类工作:

1. 打开 GPIO 和 SPI 外设时钟。
2. 设置 GPIO pin 为 alternate function, 让引脚连接到 SPI 外设。
3. 配置 SPI 模式: master/slave、full-duplex/half-duplex、frame size、CPOL/CPHA、baud rate、bit order。
4. 使能 SPI。
5. 通过 SPIx_DR 和状态位 TXE/RXNE/BSY 读写数据。

关键结论

寄存器配置不是零散动作。正确顺序通常是: 开时钟 - 配 GPIO AF - 配 SPI 控制寄存器 - 最后 enable SPI。

20 Slide 20: Block Diagram - SPIx_DR 数据寄存器

这一页开始讲 SPI 数据寄存器 SPIx_DR。

DR = Data Register, 数据寄存器。它是 CPU 和 SPI 硬件交换数据的窗口：

- CPU 写 SPIx_DR: 把要发送的数据交给 SPI。
- CPU 读 SPIx_DR: 取出 SPI 已经收到的数据。

但你不能把 DR 理解成一个简单变量。它背后连接 TXFIFO、RXFIFO 和移位寄存器。

FIFO

FIFO = First In, First Out, 先进先出队列。先进入队列的数据, 先被读出。SPI 的 TXFIFO 存待发送数据, RXFIFO 存已接收但 CPU 还没读的数据。

21 Slide 21: SPIx_DR 的读写含义

这一页说明：

- 读 SPIx_DR: 返回 RXFIFO 中还没读过的最老数据。
- 写 SPIx_DR: 把数据放到 TXFIFO 队尾, 等待发送。

这说明 SPI 内部不是“一写就立刻发完”。写 DR 只是把数据放进发送队列。硬件随后按 SCK 一个 bit 一个 bit 移出去。

关键结论

写 SPIx_DR 是“提交发送任务”；读 SPIx_DR 是“取走已接收结果”。中间的移位、产生时钟、进出 FIFO 都由 SPI 硬件完成。

22 Slide 22: RXNE 与 FRLVL - 接收 FIFO 什么时候能读

这一页讲接收相关状态位。

RXNE

RXNE = Receive Buffer Not Empty, 接收缓冲区非空。当 RXNE=1, 表示已经有接收到的数据可以读。此时 CPU 可以读 SPIx_DR。

FRLVL

FRLVL = FIFO Reception Level, 接收 FIFO 占用水平。FRLVL[1:0] 表示 RXFIFO 当前有多少数据。

易错点

读 SPIx_DR 前要看 RXNE。没有 RXNE 就读, 可能读不到有效新数据。轮询代码通常写: `while((SPIx->SR & SPI_SR_RXNE) != SPI_SR_RXNE);`。

23 Slide 23: SPIx_CR2 - 数据长度和 FIFO 阈值

这一页显示 SPIx_CR2。这个寄存器里有多和数数据帧、FIFO、中断相关的控制位。

重点字段包括：

- DS[3:0]: Data Size, 设置数据帧长度, 常用 8-bit。
- FRXTH: RXFIFO threshold, 接收 FIFO 阈值, 决定 RXNE 什么时候置位。
- TXEIE/RXNEIE: TXE/RXNE 中断使能位。

Threshold 阈值

阈值就是触发条件的门槛。比如设置 8-bit 阈值时, RXFIFO 里有 8-bit 数据就可以触发 RXNE; 设置 16-bit 阈值时, 要达到更大的接收量才认为可读。

24 Slide 24: TXE - 发送 FIFO 什么时候能写

这一页讲发送侧的管理: 写入待发送的数据要由 TXE event 管理。

TXE = Transmit Buffer Empty, 发送缓冲区空。在 STM32 SPI 语境里更直白地说: TXE=1 表示 TXFIFO 有空间, CPU 可以继续写 SPIx_DR。

轮询发送时常见代码:

```
while ((SPIx->SR & SPI_SR_TXE) != SPI_SR_TXE) {
    // wait until TXFIFO has space
}
SPIx->DR = data;
```

关键结论

TXE 不是“数据已经到 slave 了”。TXE 只是说发送 FIFO 有空间, 可以写新数据。要确认整个 SPI 传输结束, 还要看 BSY 是否清零。

25 Slide 25: FTLVL 和 TXE - 发送 FIFO 占用水平

这一页讲 FTLVL。

FTLVL = FIFO Transmission Level, 发送 FIFO 占用水平。FTLVL[1:0] 表示 TXFIFO 当前占用情况。

PPT 说: 当 TXFIFO level 小于或等于半容量时, TXE event 触发; 否则 TXE 清零, TXFIFO 被认为满。

这意味着:

- TXE=1: 可以继续写数据。
- TXE=0: 不要写, 可能覆盖或丢数据。

易错点

初学者常犯错误: 连续写很多次 SPIx_DR, 不检查 TXE。这样写不是“更快”, 而是在赌 FIFO 没满。寄存器不是垃圾桶, 满了还塞就是错。

26 Slide 26: BSY - SPI 忙标志

这一页显示 SPIx_SR 中的状态位, 重点通常包括 BSY。

BSY = Busy flag, 忙标志。当 BSY=1, 表示 SPI 正在通信或内部还没有完成最后的数据移位。

为什么发送结束后还要等 BSY 清零? 因为 TXE=1 只说明 TXFIFO 可以写, 不代表最后一个 bit 已经从线上发完。如果你过早拉高 CS, slave 可能只收到半截 frame。

关键结论

SPI 结束一次完整传输的判断通常是：数据写完，收到该收的数据，然后等待 BSY=0，再释放 CS。

27 Slide 27: Example - STM32L4 连接 L3GD20 Gyro

这一页给出例子：STM32L4 作为 SPI master, L3GD20 Gyro 作为 SPI slave。Gyro 是 gyroscope, 中文是陀螺仪, 用来测角速度, 对应 yaw、pitch、roll 等旋转轴。

图中连接包括：

- SPI2_Clock (PD1) → MEMS_SCK
- SPI2_MOSI (PD4) → MEMS_MOSI
- SPI2_MISO (PD3) ← MEMS_MISO
- GPIO PD7 → GYRO_CS

注意 CS 用普通 GPIO 控制，而不一定用硬件 NSS。很多工程里手动控制 CS 更灵活。

易错点

同一个 SPI 总线上可能还有 XL_CS、MAG_CS 等多个片选。想和 gyro 通信就拉低 GYRO_CS，其他片选保持高电平。

28 Slide 28: Processor's Pins - 从芯片引脚表找 SPI 功能

这一页展示处理器引脚图/表。目的不是让你背图，而是告诉你：STM32 一个物理引脚可以有很多功能。比如一个 pin 既可以做 GPIO，也可以做 SPI 的 SCK/MOSI/MISO。

这叫 **pin multiplexing**, 引脚复用。

要使用 SPI，必须完成两步：

1. 把 GPIO mode 设置成 **Alternate Function mode**。
2. 在 AFRL/AFRH 寄存器中选择正确 AF 编号，例如 AF5 表示某些 SPI 功能。

关键结论

一个 pin 不会自动变成 SPI。你必须在 GPIO 寄存器中明确告诉芯片：“这个 pin 现在不要当普通 GPIO，用它的 SPI alternate function。”

29 Slide 29: Alternate Mode - 把 GPIO 引脚设置为复用功能

这一页讲 GPIOx_MODER。每个 pin 用 2 bits 表示模式：

MODER bits	模式
00	Input 输入
01	Output 输出
10	Alternate Function 复用功能
11	Analog 模拟

PPT 代码：

```

GPIO->MODER &= ~(3UL << 2); // Clear bits for pin 1
GPIO->MODER |= (2UL << 2); // Set pin 1 as alternate function

```

这里 3UL 的二进制是 11，用于清除某个 pin 的两位模式位；2UL 的二进制是 10，表示 alternate function。

30 Slide 30: Which Alternate Mode - AFRL 低 8 个引脚

这一页讲 GPIO_AFRL。AFRL 管理 pin 0 到 pin 7 的 alternate function 选择，每个 pin 用 4 bits。

如果一个引脚设置成 alternate function mode，还不够。你还要选择具体是 AF0、AF1、AF2……还是 AF15。不同 AF 对应不同外设功能。

例如：

- 某个 PD1 可能 AF5 是 SPI2_SCK。
- 另一个引脚可能 AF7 是 USART。

易错点

MODER=10 只是说“我要用复用功能”；AFRL/AFRH 才说明“具体复用成哪个外设”。漏掉 AFR 配置，pin 还是不会正确连接到 SPI。

31 Slide 31: Which Alternate Mode - AFRH 高 8 个引脚

这一页讲 GPIO_AFRH。AFRH 管理 pin 8 到 pin 15。规则和 AFRL 一样：每个 pin 4 bits，选择 AF0 到 AF15。

寄存器	管理范围	每个 pin 占用
AFRL	pin 0-7	4 bits
AFRH	pin 8-15	4 bits

关键结论

GPIO 配 SPI 引脚的完整逻辑：开 GPIO 时钟 - MODER 设 AF mode - AFRL/AFRH 选正确 AF 编号 - 配速度/上下拉/输出类型（按需要）。

32 Slide 32: SPI Initialization - 开 SPI 时钟并复位

这一页进入 SPI_Init 代码。第一段做的是：根据传入的是 SPI1/SPI2/SPI3，打开对应外设时钟，并做 reset。

核心操作：

- `RCC->APB2ENR |= RCC_APB2ENR_SPI1EN;` 打开 SPI1 时钟。
- `RCC->APB1ENR1 |= RCC_APB1ENR1_SPI2EN;` 打开 SPI2 时钟。
- 先置 reset 位，再清 reset 位，确保外设处于干净初始状态。
- 最后 `SPIx->CR1 &= ~SPI_CR1_SPE;` 先 disable SPI。

关键结论

修改 SPI 配置前先 disable。外设正在运行时边跑边改配置，可能导致半帧数据、错误时钟或状态机异常。

33 Slide 33: SPI Initialization - 通信模式与数据帧格式

这一页配置 full-duplex、half-duplex、receive-only 和 data frame format。

RXONLY

RXONLY=0 表示 full-duplex，发送和接收都允许。RXONLY=1 表示 receive-only。

BIDIMODE

BIDIMODE=0 表示 2-line unidirectional data mode，也就是 MOSI/MISO 两根线。BIDIMODE=1 表示 1-line bidirectional data mode，也就是半双工一线模式。

BIDIOE

在 bidirectional mode 里控制方向：输出使能表示 transmit-only，输出禁用表示 receive-only。

DS

DS 设置 data frame size。代码里设置 0111 表示 8-bit。

易错点

Full-duplex、half-duplex、receive-only 不是口号，它们最后都会落到 RXONLY, BIDIMODE, BIDIOE 这些控制位上。

34 Slide 34: SPI Initialization - bit order, CPHA, CPOL, baud rate

这一页配置 SPI 最核心的通信格式。

Bit order

LSBFIRST=0: MSB first。代码清除 LSBFIRST，表示最高位先传。

Clock phase

CPHA=0: 第一个 clock transition 是第一个 data capture edge。代码清除 CPHA，表示 1st edge capture。

Clock polarity

CPOL=0: SCK idle 时为 0。代码清除 CPOL，表示 polarity low。

Baud rate control

BR[2:0] 设置 SCK 分频：

BR bits	SCK
000	$f_{PCLK}/2$
001	$f_{PCLK}/4$
010	$f_{PCLK}/8$
011	$f_{PCLK}/16$
100	$f_{PCLK}/32$
101	$f_{PCLK}/64$
110	$f_{PCLK}/128$
111	$f_{PCLK}/256$

PPT 代码中 `3U<<3` 设置 `BR=011`。如果 $f_{PCLK} = 80\text{ MHz}$ ，则：

$$f_{SCK} = 80\text{ MHz}/16 = 5\text{ MHz}$$

35 Slide 35: SPI Initialization - master, RXNE 阈值, enable SPI

这一页完成初始化。

MSTR

`MSTR=1` 表示设置为 master。Master 的职责是产生 SCK，并主动开始传输。

FRXTH

`FRXTH` 设置 `RXNE` 的阈值。PPT 说明：如果 `FRXTH` set, `RXFIFO` 达到 1/4 (8-bit) 时 `RXNE` 置位；如果 `FRXTH` cleared, `RXFIFO` 达到 1/2 (16-bit) 时 `RXNE` 置位。

SPE

`SPE=1` 表示 SPI enable。所有配置完成后，最后使能 SPI。

关键结论

初始化最后一步才开 `SPE`。前面所有配置都是“搭机器”，`SPE=1` 才是“开机运行”。

36 Slide 36: SPI Write - 写数据函数逐行解释

这一页给出 `SPI_Write` 函数。表面叫 write，但因为 SPI 是 full-duplex，写的同时也会接收数据，所以函数里也读 `rxBuffer`。

核心代码逻辑：

```
for (i = 0; i < size; i++) {
    while ((SPIx->SR & SPI_SR_TXE) != SPI_SR_TXE);
    SPIx->DR = txBuffer[i];

    while ((SPIx->SR & SPI_SR_RXNE) != SPI_SR_RXNE);
    rxBuffer[i] = SPIx->DR;
}
while ((SPIx->SR & SPI_SR_BSY) == SPI_SR_BSY);
```

逐句解释：

- 等 `TXE=1`：发送 FIFO 有空间。

- 写 DR：提交一个 byte/frame 要发送。
- 等 RXNE=1：对应接收数据已经到达。
- 读 DR：取走收到的数据，避免 RXFIFO 堆积。
- 最后等 BSY=0：确认 SPI 线上最后一位也发完。

易错点

SPI write 里读 DR 不是多余动作。Full-duplex 中每发一个 frame 都会收一个 frame。如果你不读 RXFIFO，可能导致接收溢出或后续状态异常。

37 Slide 37: SPI Read - 为什么读也要写 dummy byte

这一页是 SPI 初学者最容易误解的一页。

SPI master 要读 slave，仍然必须产生 SCK。而 SCK 只有 master 在发送 frame 时才会产生。因此 master 想读数据时，通常要写一个 dummy byte，例如 0xFF，目的不是给 slave 真正的数据，而是**制造时钟**。

核心逻辑：

```
while ((SPIx->SR & SPI_SR_TXE) != SPI_SR_TXE);
SPIx->DR = 0xFF; // dummy byte, creates clock
while ((SPIx->SR & SPI_SR_RXNE) != SPI_SR_RXNE);
rxBuffer[i] = SPIx->DR;
```

关键结论

SPI 里 master 读 slave 的本质：master 发送 dummy data 产生 SCK，slave 借这个 SCK 把数据移出来，master 再读 DR。

38 Slide 38: Interrupt - SPI 也可以用中断

这一页是中断章节的过渡。前面写/读函数用的是 polling：CPU 一直 while 等 TXE/RXNE/BSY。Polling 简单，但 CPU 会浪费时间。

SPI 也可以用 interrupt：当 TXE 或 RXNE 发生时，硬件触发中断，CPU 跳到 ISR 处理。

方式	优点	缺点
Polling	简单直接	CPU 忙等，效率低
Interrupt	CPU 可做别的事，事件来了再处理	程序结构更复杂
DMA	大块数据效率最高	配置更复杂

39 Slide 39: Polling vs Interrupt - SPI 中断使能路径

这一页显示两个事件：

- **TXE**: TXFIFO ready to be loaded, 发送 FIFO 有空间。
- **RXNE**: Data received in RXFIFO, 接收 FIFO 中有数据。

要使用 SPI 中断，必须同时配置两层：

1. 在 SPIx_CR2 里打开 TXEIE 或 RXNEIE。
2. 在 NVIC 里 NVIC_EnableIRQ(SPIx_IRQn)。

易错点

只设置 TXEIE/RXNEIE 但没有 NVIC enable, CPU 不会进入 ISR。只 NVIC enable 但 SPI 没开对应中断事件, 也不会有 SPI 中断。两边都要打开。

40 Slide 40: Interrupt Handler - SPI 中断服务函数

这一页给出 SPI ISR 的结构: 检查是 TXE 还是 RXNE, 然后分别处理。

典型逻辑:

```
void SPIx_IRQHandler(void) {  
    if (SPIx->SR & SPI_SR_RXNE) {  
        // read SPIx->DR, store received data  
    }  
    if (SPIx->SR & SPI_SR_TXE) {  
        // write next data to SPIx->DR  
    }  
}
```

ISR 里不要做很重的工作。中断服务函数应该快速处理状态位、搬运数据、更新索引, 然后退出。复杂计算放到主程序做。

关键结论

SPI 中断处理的本质: RXNE 来了就读 DR, TXE 来了就写下一个 DR。状态位告诉 CPU 现在该做哪一步。

总复习：Lecture 20 必须掌握的知识点

1. SPI 的一句话定义

SPI 是一种同步串行通信接口，master 通过 SCK 控制传输节奏，通过 MOSI/MISO 与 slave 交换数据，通过 NSS/CS 选择具体 slave。

2. 四根线

线	作用	方向
SCK	时钟	Master 到 Slave
MOSI	主发从收	Master 到 Slave
MISO	从发主收	Slave 到 Master
NSS/CS	片选，低有效	Master 到 Slave

3. 三种通信模式

模式	中文	特点
Full-duplex	全双工	MOSI/MISO 两根线，同时收发
Half-duplex	半双工	一根数据线，轮流收发
Simplex	单工	单方向通信

4. SPI clock mode

- CPOL 决定 SCK idle 是高还是低。
- CPHA 决定第一个还是第二个边沿采样。
- CPOL/CPHA 必须按 slave datasheet 配。

5. 状态位

状态位	中文	什么时候用
TXE	发送缓冲区空	写 DR 前等待
RXNE	接收缓冲区非空	读 DR 前等待
BSY	忙标志	结束通信前等待清零
FRLVL	RXFIFO level	判断接收 FIFO 占用
FTLVL	TXFIFO level	判断发送 FIFO 占用

6. 编程顺序

1. 打开 GPIO 和 SPI 时钟。
2. GPIO 设置成 alternate function。
3. AFRL/AFRH 选择正确 AF。
4. 配置 SPIx_CR1/CR2: master, CPOL, CPHA, baud rate, frame size, bit order。
5. 设置 SPE=1 使能 SPI。
6. 读写时检查 TXE/RXNE/BSY。

易错点

最容易丢分/出错的点：

- 忘记 TX/RX 不是 UART 里的名字，SPI 要看 MOSI/MISO。
- CPOL/CPHA 和 slave 不匹配。
- 写 DR 前不等 TXE。
- 读 DR 前不等 RXNE。
- 传完立刻释放 CS，不等 BSY=0。
- 想“读 SPI”却不写 dummy byte，导致没有 SCK，slave 根本没有机会输出数据。